

Лабораторная работа №5

Вероятностные алгоритмы проверки чисел на простоту

Кашкин Иван Евгеньевич

Содержание

Цель работы	1
Задание.....	1
Вероятностные алгоритмы проверки на простоту.....	1
Выполнение лабораторной работы.....	2
Реализация алгоритмов	2
Анализ результатов.....	4
Выводы.....	4

Цель работы

Изучение и реализация вероятностных алгоритмов проверки чисел на простоту: теста Ферма, теста Соловья-Штрассена и теста Миллера-Рабина.

Задание

Реализовать на языке Julia три вероятностных алгоритма проверки чисел на простоту: -

Тест Ферма - Тест Соловья-Штрассена

- Тест Миллера-Рабина - # Теоретическая часть

Вероятностные алгоритмы проверки на простоту

Вероятностные алгоритмы используют генератор случайных чисел и дают не гарантированно точный ответ. После нескольких независимых выполнений теста вероятность ошибки становится пренебрежимо малой.

1. Тест Ферма

Основан на малой теореме Ферма: для простого числа p и произвольного числа a , $1 \leq a \leq p - 1$, выполняется сравнение:

$$a^{p-1} \equiv 1 \pmod{p}$$

2. Тест Соловья-Штрассена

Основан на критерии Эйлера и вычислении символа Якоби. Для простого числа n и любого целого a , взаимно простого с n , выполняется:

$$a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n}$$

3. Тест Миллера-Рабина

Основан на разложении $n - 1 = 2^s \cdot r$ и проверке последовательности степеней. Если n простое, то для любого $a \geq 2$ выполняется одно из условий: $-a^r \equiv 1 \pmod{n}$ - $a^{2^j \cdot r} \equiv -1 \pmod{n}$ для некоторого j , $0 \leq j \leq s - 1$

4. Символ Якоби

Символ Якоби $\left(\frac{a}{n}\right)$ является обобщением символа Лежандра на нечетные составные модули и вычисляется рекурсивно на основе свойств мультипликативности и квадратичной взаимности.

Выполнение лабораторной работы

Реализация алгоритмов

1. Вычисление символа Якоби

```
function jacobi_symbol(a::Int, n::Int)
    if n % 2 == 0 || n < 3
        throw(ArgumentError("n должно быть нечётным и >= 3"))
    end

    a = a % n
    g = 1

    while a != 0
        while a % 2 == 0
            a ÷= 2
            if n % 8 == 3 || n % 8 == 5
                g = -g
            end
        end

        a, n = n, a

        if a % 4 == 3 && n % 4 == 3
            g = -g
        end

        a = a % n
    end
end
```

```
    return n == 1 ? g : 0
end
```

2. Тест Ферма

```
function fermat_test(n::Int, k::Int=5)
    if n < 5
        throw(ArgumentError("n должно быть >= 5"))
    end
    if n % 2 == 0
        return "Составное"
    end

    for _ in 1:k
        a = rand(2:(n-2))
        if powermod(a, n-1, n) != 1
            return "Составное"
        end
    end
    return "Вероятно простое"
end
```

3. Тест Соловья-Штрассена

```
function solovay_strassen_test(n::Int, k::Int=5)
    if n < 5
        throw(ArgumentError("n должно быть >= 5"))
    end
    if n % 2 == 0
        return "Составное"
    end

    for _ in 1:k
        a = rand(2:(n-2))
        r = powermod(a, (n-1)÷2, n)

        if r != 1 && r != n-1
            return "Составное"
        end

        s = jacobi_symbol(a, n)
        if r % n != s % n
            return "Составное"
        end
    end
    return "Вероятно простое"
end
```

4. Тест Миллера-Рабина

```
function miller_rabin_test(n::Int, k::Int=5)
    if n < 5
```

```

        throw(ArgumentError("n должно быть >= 5"))
    end
    if n % 2 == 0
        return "Составное"
    end

    s = 0
    r = n - 1
    while r % 2 == 0
        s += 1
        r ÷= 2
    end

    for _ in 1:k
        a = rand(2:(n-2))
        y = powermod(a, r, n)

        if y != 1 && y != n-1
            j = 1
            while j < s && y != n-1
                y = powermod(y, 2, n)
                if y == 1
                    return "Составное"
                end
                j += 1
            end
            if y != n-1
                return "Составное"
            end
        end
    end
    return "Вероятно простое"
end

```

Анализ результатов

- Все три алгоритма корректно определяют простые числа (5, 7, 11, 17, 19, 23, 29)
- Составные числа (15, 21, 49) правильно идентифицируются всеми тестами
- Число Кармайкла 561 демонстрирует ограничения теста Ферма: он ошибочно определяет его как "Вероятно простое", в то время как тесты Соловья-Штрассена и Миллера-Рабина правильно идентифицируют его как составное

Выводы

В ходе выполнения лабораторной работы были успешно реализованы и протестированы три вероятностных алгоритма проверки чисел на простоту: - Тест Ферма - самый простой алгоритм, основанный на малой теореме Ферма, но имеющий ограничения (не определяет числа Кармайкла) - Тест Соловья-Штрассена - более надежный алгоритм, использующий символ Якоби, который успешно определяет числа Кармайкла - Тест

Миллера-Рабина - наиболее надежный из рассмотренных алгоритмов, широко применяемый на практике благодаря высокой точности и эффективности